

数据库系统 Ch10 —数据库索引

详细学习笔记

基于：数据库系统 PPT Ch10 DataBase Index (85 页)

2026 年 6 月 21 日

目录

1 索引基本概念 (Index Basic Concepts) [p.1–5]	2
1.1 什么是数据库索引? [p.1]	2
1.2 索引示例：百万级数据查询加速 [p.2–3]	2
1.3 索引基本结构 [p.4]	2
1.4 索引原理示意 [p.5]	2
2 索引评估指标 (Index Evaluation Metrics) [p.6]	3
3 有序索引 (Ordered Indices) [p.7–19]	3
3.1 主索引与辅助索引 [p.7]	3
3.2 稠密索引 vs 稀疏索引 (Dense vs Sparse Index) [p.8–12]	3
3.2.1 稠密索引 (Dense Index) [p.8–9]	3
3.2.2 稀疏索引 (Sparse Index) [p.10–12]	3
3.3 辅助索引 (Secondary Index) [p.12, p.18]	4
3.4 索引的更新开销 [p.13]	4
3.5 多级索引 (Multilevel Index) [p.14–15, p.19–20]	4
3.6 单级索引的删除与插入 [p.16–17]	5
3.6.1 删除操作 (Deletion) [p.16]	5
3.6.2 插入操作 (Insertion) [p.17]	5
4 B 树与 B+ 树对比 (B-Tree vs B+Tree) [p.20–24]	5
4.1 二叉树用于索引? [p.20]	5
4.2 B 树特点 [p.21]	5

4.3	B+ 树特点 [p.22]	6
4.4	关系数据库为何选择 B+ 树? [p.23–24]	6
5	B+ 树索引结构 (B+Tree Index Structure) [p.25–46]	6
5.1	B+ 树的优势 [p.25]	6
5.2	B+ 树的形式化定义 [p.27–30]	7
5.2.1	基本性质 [p.27]	7
5.2.2	节点结构 [p.28]	7
5.2.3	叶子节点性质 [p.29]	7
5.2.4	非叶节点性质 [p.30]	7
5.3	B+ 树的高度与查询复杂度 [p.32, p.37]	7
5.3.1	树高度分析 [p.32]	7
5.3.2	查询复杂度量化分析 [p.37]	8
5.4	B+ 树查询算法 (B+Tree Query) [p.36]	8
5.5	B+ 树插入与分裂 (Insertion & Splitting) [p.38–42]	8
5.5.1	插入算法概要 [p.38]	8
5.5.2	叶子节点分裂 [p.39]	9
5.5.3	非叶节点分裂 [p.42]	9
5.6	B+ 树删除与合并 (Deletion & Merging) [p.43–46]	9
5.6.1	删除处理策略 [p.43–46]	9
5.6.2	删除操作总结 [p.46]	10
5.7	InnoDB 引擎中 B+ 树参数 n 的计算 [p.33–35]	10
5.7.1	参数 n 的影响因素 [p.33]	10
5.7.2	单个键的存储开销 [p.34]	10
5.7.3	单页键容量与树高 [p.34–35]	10
6	MySQL 中的索引 (MySQL Index) [p.47–54]	11
6.1	索引的逻辑分类 [p.47]	11
6.2	单列索引与复合索引 [p.48–49]	11
6.2.1	单列索引 (Single-Column Index) [p.48]	11
6.2.2	复合索引 / 多列索引 (Composite Index) [p.49]	11
6.3	索引的查看与创建 [p.50–52]	12
6.3.1	查看索引 [p.50]	12

6.3.2	创建索引 [p.51–52]	12
6.4	索引失效的场景 [p.53]	12
6.5	索引的正确使用 [p.54]	13
6.5.1	索引的优点 [p.54]	13
6.5.2	索引的缺点 [p.54]	13
7	散列索引 (Hash Index) [p.55–77]	13
7.1	散列基本概念 [p.55]	13
7.2	散列函数示例 [p.56–58]	13
7.2.1	示例：以 dept_name 为键的散列文件 [p.56–57]	13
7.2.2	理想散列函数 [p.58]	13
7.3	桶溢出 (Bucket Overflow) [p.59–60]	14
7.3.1	溢出原因 [p.59]	14
7.3.2	溢出处理方式 [p.60]	14
7.4	散列索引 (Hash Index) [p.61–62]	14
7.5	静态散列的问题 (Static Hashing Problems) [p.63]	14
7.6	可扩展散列 (Extendable Hashing) [p.64–77]	15
7.6.1	基本原理 [p.64]	15
7.6.2	数据结构 [p.65–66]	15
7.6.3	桶分裂算法 [p.67]	15
7.6.4	删除与合并 [p.68]	15
7.6.5	可扩展散列示例 [p.70–76]	16
7.6.6	可扩展散列的优缺点 [p.77]	16
8	有序索引 vs 散列索引对比 [p.78]	16
9	位图索引 (Bitmap Index) [p.79–83]	17
9.1	基本概念 [p.79]	17
9.2	位图索引结构 [p.80]	17
9.3	位图操作与查询 [p.81]	17
9.4	位图索引的空间效率 [p.82]	17
9.5	位图计算优化 [p.83]	18
10	SQL 索引语法 (SQL Index Syntax) [p.84]	18

11 章末总结：核心考点速查表

18

1 索引基本概念 (Index Basic Concepts) [p.1–5]

1.1 什么是数据库索引? [p.1]

- **定义:** 数据库索引 (Database Index) 是一种用于加速数据检索的数据结构, 它通过预先排序表中的一列或多列值, 并建立指向数据行的指针, 从而避免全表扫描 (Full Table Scan), 显著提高查询效率 [p.1]
- **类比 (Analogy):** 图书馆的图书目录 (Author Catalog), 无需遍历所有书架即可找到目标 [p.1]
- **索引机制**用于加速对所需数据的访问 (Indexing mechanisms used to speed up access to desired data) [p.1]

1.2 索引示例: 百万级数据查询加速 [p.2–3]

- 向 student 表插入 100 万条记录的存储过程 (Stored Procedure), 使用批量插入和事务优化 [p.2]
- 无索引查询: `SELECT * FROM student WHERE s_name='Student_9999999'` [p.3]
- 创建索引: `CREATE INDEX idx_s_name ON student(s_name);` [p.3]
- 查询速度提升 20 倍以上 (Speed improvement: >20×) [p.3]

1.3 索引基本结构 [p.4]

- **搜索码 (Search Key):** 用于在文件中查找记录的属性或属性集合 (attribute to set of attributes used to look up records in a file) [p.4]
- **索引项 (Index Entry):** 索引文件由记录 (称为索引项) 组成, 形式为:



[p.4]

- 索引文件通常远小于原始文件 (Index files are typically much smaller than the original file) [p.4]
- **两种基本索引类型** (Two basic kinds of indices) [p.4]:
 1. **有序索引 (Ordered Indices):** 搜索码按排序顺序存储 (search keys are stored in sorted order)
 2. **散列索引 (Hash Indices):** 搜索码使用散列函数均匀分布到各个”桶” (buckets) 中 (search keys are distributed uniformly across ”buckets” using a ”hash function”)

1.4 索引原理示意 [p.5]

- 索引通过 search-key 到 pointer 的映射, 实现对数据页 (Page) 的快速定位
- 例: 按字母顺序排列的图书索引, 每个条目指向对应图书所在的页码

2 索引评估指标 (Index Evaluation Metrics) [p.6]

- 访问类型 (Access Types) 是否被高效支持 [p.6]:
 - 等值查询 (Point Query / Equality Search): 具有指定属性值的记录 (records with a specified value)
 - 范围查询 (Range Query): 属性值落在指定范围内的记录 (records with attribute value in a specified range)
- 访问时间 (Access Time): 查找特定记录所需的时间 [p.6]
- 插入时间 (Insertion Time): 插入新记录并更新索引所需的时间 [p.6]
- 删除时间 (Deletion Time): 删除记录并更新索引所需的时间 [p.6]
- 空间开销 (Space Overhead): 索引结构占用的额外存储空间 [p.6]

3 有序索引 (Ordered Indices) [p.7–19]

3.1 主索引与辅助索引 [p.7]

- 有序索引: 索引项按搜索码值排序存储, 如: 图书馆作者目录 [p.7]
- 主索引 (Primary Index / Clustering Index, 聚集索引): 在顺序排序的文件中, 搜索码指定文件顺序的索引 [p.7]
 - 也称聚簇索引 (Clustering Index)
 - 主索引的搜索码通常是但不一定是主键 (Primary Key) [p.7]
- 辅助索引 (Secondary Index / Non-clustering Index): 搜索码指定的顺序与文件顺序不同的索引 [p.7]
- 索引顺序文件 (Index-Sequential File): 具有主索引的有序顺序文件 [p.7]

3.2 稠密索引 vs 稀疏索引 (Dense vs Sparse Index) [p.8–12]

3.2.1 稠密索引 (Dense Index) [p.8–9]

- 定义: 文件中每个搜索码值都有一个对应的索引记录 (Index record appears for every search-key value in the file) [p.8]
- 例: instructor 关系上的 ID 属性索引 [p.8]
- 在 dept_name 上建立稠密索引, instructor 文件按 dept_name 排序 [p.9]
- 优点: 查找效率高, 直接定位到目标记录
- 缺点: 占用空间大; 维护开销高 (插入/删除需更新索引)

3.2.2 稀疏索引 (Sparse Index) [p.10–12]

- 定义: 仅对部分搜索码值包含索引记录 (contains index records for only some search-key values) [p.10]

- **适用条件：**记录按搜索码顺序存放 [p.10]
- **查找过程** (To locate a record with search-key value K): [p.10]
 1. 找到搜索码值为小于 K 的最大值对应索引记录
 2. 从该索引记录指向的位置开始顺序搜索文件
- **与稠密索引对比** [p.11]:
 - 空间和维护开销更少 (Less space and maintenance overhead)
 - 查找速度通常慢于稠密索引 (Generally slower than dense index for locating records)
- **好的折中方案** (Good tradeoff): 为文件中每个块 (Block) 建立一个索引条目, 对应块中最小搜索码值 [p.11]

3.3 辅助索引 (Secondary Index) [p.12, p.18]

- 辅助索引记录指向一个存储桶 (Bucket), 该存储桶包含指向所有具有该特定搜索码值的实际记录的指针 [p.12]
- **辅助索引必须是稠密索引** (Secondary indices have to be dense) [p.12]
- **例:** instructor 关系中按 salary 字段建立辅助索引 [p.12]
- **应用场景:** 在主索引搜索码以外的字段上进行查询 [p.18]
 - 例 1: instructor 按 ID 顺序存储, 需要查询特定 department 的所有 instructor
 - 例 2: 查找指定 salary 或 salary 在某范围内的所有 instructor

3.4 索引的更新开销 [p.13]

- 索引在搜索记录时提供巨大好处 [p.13]
- **但是:** 更新索引会给数据库修改带来开销——文件被修改时, 文件上的每个索引都必须更新 [p.13]
- 使用主索引的顺序扫描 (Sequential Scan) 是高效的, 但使用辅助索引的顺序扫描代价高昂 [p.13]
 - 因为主索引的数据物理有序, 而二级索引需要频繁跳转读取
 - 每次记录访问可能从磁盘获取一个新的块
 - 磁盘块读取约需 5–10 ms, 而内存访问约需 100 ns (差距约 5 万–10 万倍) [p.13]

3.5 多级索引 (Multilevel Index) [p.14–15, p.19–20]

- **问题:** 如果主索引无法完全载入内存, 访问成本变得高昂 [p.14]
- **解决:** 将磁盘上的主索引视为顺序文件, 在其上构造稀疏索引 [p.14]
 - **外层索引** (Outer Index): 主索引的稀疏索引, 存放在内存中
 - **内层索引** (Inner Index): 主索引文件本身
- 如果外层索引仍然太大无法放入内存, 可以创建更多层的索引 [p.14]
- 所有层级的索引在文件插入或删除时都需要更新 [p.14]

- 核心思路：将索引分层存储，解决单 block 空间有限无法容纳所有索引的问题 [p.19–20]

3.6 单级索引的删除与插入 [p.16–17]

3.6.1 删除操作 (Deletion) [p.16]

- 如果被删记录是文件中具有该搜索码值的唯一记录，则从索引中删除此搜索码 [p.16]
- 稠密索引：搜索码的删除类似于文件记录的删除 [p.16]
- 稀疏索引：
 - 如果索引中存在该搜索码的条目，则用文件中下一个搜索码值替换该条目
 - 如果下一个搜索码值已经有索引条目，则直接删除原条目（不替换） [p.16]
- 多级索引的插入和删除算法是单级算法的简单扩展 [p.17]

3.6.2 插入操作 (Insertion) [p.17]

- 使用要插入记录中的搜索码值执行查找 [p.17]
- 稠密索引：如果搜索码值不在索引中，则插入 [p.17]
- 稀疏索引：如果索引为文件的每个块存储一个条目，且未创建新块，则无需更改索引。如果创建了新块，将新块中出现的第一个搜索码值插入索引 [p.17]

4 B 树与 B+ 树对比 (B-Tree vs B+Tree) [p.20–24]

4.1 二叉树用于索引？ [p.20]

- 节点结构：每个节点最多 2 个子节点（左/右） [p.20]
- 所有节点均可存储数据 [p.20]
- 叶子节点无特殊连接 [p.20]
- 范围查询：平均 $O(\log n)$ ，但可能退化为 $O(n)$ [p.20]
- 插入/删除：可能频繁调整结构，代价高 [p.20]
- 适用场景：内存中的高频随机访问（如 MySQL 的 MyISAM 引擎非聚簇索引的早期实现） [p.20]

4.2 B 树特点 [p.21]

- 树内每个节点都存储数据 [p.21]
- 叶子节点之间无指针连接 [p.21]
- 需中序遍历，效率中等 [p.21]
- 适用场景：文件系统（NTFS、ReiserFS）、非关系型数据库（MongoDB 索引） [p.21]
- 适合读写混合场景，所有节点存数据，适合随机读写 [p.21]

4.3 B+ 树特点 [p.22]

- 数据只出现在叶子节点 (Data only at leaf nodes) [p.22]
- 所有叶子节点有链指针连接 (All leaf nodes linked by pointers) [p.22]
- 直接遍历叶子链表，范围查询极高效 [p.22]
- 适用场景：关系型数据库索引 (MySQL InnoDB、Oracle)、大数据存储 (HBase) [p.22]
- 优势：范围查询高效、磁盘友好、更高填充率、适合顺序扫描 [p.22]

4.4 关系数据库为何选择 B+ 树? [p.23–24]

- 每个节点存储更多 **Key**: B 树的每个节点存储 key+data，而磁盘页大小有限（如 InnoDB 的 16KB 页）。如果 data 很大，每个节点能存的 key 数量很小，导致树高度增加，增加磁盘 I/O 次数。B+ 树仅在叶子节点存储数据，非叶子节点只存 key，大大增加单节点 Key 数，降低树高度 [p.24]
- 范围查询高效: B+ 树叶子节点的链表指针支持高效范围遍历，只需找到范围起点后沿链表扫描即可 [p.24]
- B 树 vs B+ 树对比表 [p.23]:

	B 树	B+ 树
数据存储	所有节点均存储数据	仅叶子节点存储数据
叶子连接	无指针连接	叶子节点有链指针
范围查询	需中序遍历，中等效率	遍历叶子链表，极高效率
填充率	较低	更高

5 B+ 树索引结构 (B+Tree Index Structure) [p.25–46]

5.1 B+ 树的优势 [p.25]

- B+ 树索引是索引顺序文件的替代方案 [p.25]
- 索引顺序文件的缺点:
 - 文件增长时性能下降，因为产生大量溢出块 (Overflow Blocks)
 - 需要定期对整个文件进行重组 (Periodic reorganization)
- B+ 树的优势:
 - 面对插入和删除时，通过小的、局部的更改自动重组自身 (automatically reorganizes itself with small, local changes)
 - 不需要重组整个文件来维持性能
- B+ 树的（次要）缺点: 额外的插入和删除开销，空间开销 [p.25]
- 结论: B+ 树的优点远超缺点，B+ 树被广泛使用 (used extensively) [p.25]

5.2 B+ 树的形式化定义 [p.27–30]

5.2.1 基本性质 [p.27]

B+ 树是有根树，满足以下性质：

- 所有从根到叶的路径等长 (All paths from root to leaf are of the same length) [p.27]
- 非根非叶节点有 $\lceil n/2 \rceil$ 到 n 个子节点 (n 为每个节点的最大容量) [p.27]
- 叶子节点有 $\lceil (n-1)/2 \rceil$ 到 $n-1$ 个值 [p.27]
- 特殊情况 [p.27]:
 - 若根不是叶子，至少有 2 个子节点
 - 若根是叶子（树中没有其他节点），可以有 0 到 $n-1$ 个值

5.2.2 节点结构 [p.28]

- 典型非叶节点的结构：



- K_i 是搜索码值 (search-key values)
- P_i 是指向子结点的指针（非叶节点），或指向记录/记录桶的指针（叶子节点）
- 节点中的搜索码有序： $K_1 < K_2 < K_3 < \dots < K_{n-1}$ [p.28]

5.2.3 叶子节点性质 [p.29]

- 对于 $i = 1, 2, \dots, n-1$ ，指针 P_i 指向搜索码值为 K_i 的文件记录 [p.29]
- 若 L_i, L_j 是叶子节点且 $i < j$ ，则 L_i 的搜索码值都小于等于 L_j 的搜索码值 [p.29]
- P_n 指向按搜索码顺序的下一个叶子节点（链表指针） [p.29]

5.2.4 非叶节点性质 [p.30]

- 非叶节点形成叶子节点上的多级稀疏索引 (Multi-level Sparse Index) [p.30]
- 对于有 n 个指针的非叶节点：
 - P_1 指向的子树中所有搜索码都小于 K_1
 - 对于 $2 \leq i \leq n-1$ ， P_i 指向的子树中所有搜索码 $\geq K_{i-1}$ 且 $< K_i$
 - P_n 指向的子树中所有搜索码 $\geq K_{n-1}$

5.3 B+ 树的高度与查询复杂度 [p.32, p.37]

5.3.1 树高度分析 [p.32]

- 由于节点间的连接通过指针实现，“逻辑上”接近的块不一定“物理上”接近 [p.32]
- 非叶层形成稀疏索引的层次结构

- B+ 树包含相对较少的层级 [p.32]:
 - 根下一层至少有 $2 \cdot \lceil n/2 \rceil$ 个值
 - 下一层至少有 $2 \cdot \lceil n/2 \rceil \cdot \lceil n/2 \rceil$ 个值
 - 以此类推
- 若文件中有 K 个搜索码值, 树高不超过:

$$\left\lceil \log_{\lceil n/2 \rceil}(K) \right\rceil$$

[p.32]

- 搜索可高效进行; 插入删除也可在对数时间内处理 [p.32]

5.3.2 查询复杂度量化分析 [p.37]

- 树高: $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$ [p.37]
- 节点通常与磁盘块大小相同 (典型 4KB), n 约等于 100 (每索引项 40 字节) [p.37]
- 对于 100 万搜索码值, $n = 100$:

最多访问 $\log_{50}(1,000,000) = 4$ 个节点

- 对比平衡二叉树: 100 万键值需访问约 $\log_2(1,000,000) \approx 20$ 个节点 [p.37]
- 每次节点访问可能需要一次磁盘 I/O, 约 20ms, 差距巨大 [p.37]

5.4 B+ 树查询算法 (B+Tree Query) [p.36]

- 查找具有搜索码值 V 的记录 [p.36]:
 1. 设 $C = \text{root}$
 2. 当 C 不是叶子节点时:
 - 设 i 为满足 $V \leq K_i$ 的最小值
 - 若不存在这样的 i , 设 $C =$ 节点中最后一个非空指针
 - 否则, 若 $V = K_i$ 则设 $C = P_{i+1}$, 否则设 $C = P_i$
 3. 设 i 为满足 $K_i = V$ 的最小值
 4. 若存在这样的 i , 沿指针 P_i 找到所需记录
 5. 否则不存在搜索码值为 k 的记录

5.5 B+ 树插入与分裂 (Insertion & Splitting) [p.38–42]

5.5.1 插入算法概要 [p.38]

1. 找到搜索码值应出现的叶子节点
2. 若搜索码值已在叶子节点中:
 - 将记录添加到文件
 - 必要时向桶添加指针

3. 若搜索码值不在叶子节点中:

- 将记录添加到主文件（必要时创建桶）
- 若叶子节点有空间，在叶子节点中插入 (key-value, pointer) 对
- 否则，分裂节点 (Split the node)

5.5.2 叶子节点分裂 [p.39]

- 将 n 个 (search-key value, pointer) 对（包括被插入的）按排序顺序排列 [p.39]
- 前 $\lceil n/2 \rceil$ 个放入原节点，其余的放入新节点
- 设新节点为 p ， k 为 p 中最小键值，在父节点中插入 (k, p)
- 如果父节点已满，分裂父节点并向上传播 (propagate the split further up)
- 分裂向上传播直到遇到未满节点；最坏情况下根节点可能分裂，树高度增加 1 [p.39]
- 例：插入 Adams 后，叶子节点分裂；将 (Califieri, pointer-to-new-node) 插入父节点 [p.39–40]

5.5.3 非叶节点分裂 [p.42]

- 将键 (k, p) 插入已满内部节点 N 时 [p.42]:
 1. 将 N 复制到内存区域 M ，预留 $n + 1$ 指针和 n 键的空间
 2. 将 (k, p) 插入 M
 3. 将 $P_1, K_1, \dots, K_{\lceil n/2 \rceil - 1}, P_{\lceil n/2 \rceil}$ 从 M 复制回节点 N
 4. 将 $P_{\lceil n/2 \rceil + 1}, K_{\lceil n/2 \rceil + 1}, \dots, K_n, P_{n+1}$ 从 M 复制到新分配的节点 N'
 5. 将 $(K_{\lceil n/2 \rceil}, N')$ 插入父节点
- 关键区别：非叶节点分裂时，中间键值上升到父节点，本身不保留在子节点中 [p.42]

5.6 B+ 树删除与合并 (Deletion & Merging) [p.43–46]

5.6.1 删除处理策略 [p.43–46]

B+ 树删除后节点可能进入欠满 (Under-full) 状态，需采取以下策略之一：

1. 借调 (Borrow): 从左兄弟节点或右兄弟节点借一个值
 - 例：删除 Singh 和 Wu 后，从左侧兄弟节点借 Kim [p.44]
 - 父节点中的搜索码值因此改变 [p.44]
2. 合并 (Merge): 与兄弟节点合并
 - 例：删除 Gold 后，含 Gold 和 Katz 的节点欠满，与兄弟节点合并 [p.45]
 - 父节点变得欠满，再与父节点的兄弟节点合并
 - 合并时，父节点中分隔两个节点的值被拉下来 (pulled down) [p.45]
 - 根节点最终只有一个子节点，被删除，子节点成为新根 [p.45]

5.6.2 删除操作总结 [p.46]

- 合并兄弟节点是 B+ 树平衡性维护的核心操作，避免节点 Under-Full [p.46]
- 递归调整父节点，确保树高均匀，维持 $O(\log n)$ 查询效率 [p.46]
- 删除 (search-key, pointer) 从叶子节点：
 - 若无存储桶关联或桶已空时执行此操作
 - 若删除后节点条目过少且可与兄弟节点合并至单节点，执行合并 [p.46]
 - 若不能合并，则在节点与兄弟节点间重新分配指针，使两者均满足最小条目要求；更新父节点中的搜索码值 [p.46]
- 节点删除操作可能向上级联，直至遇到满足至少 $\lceil n/2 \rceil$ 个指针的节点 [p.46]
- 根节点特殊处理：若删除后根节点仅剩一个指针，则删除该根节点，其唯一子节点晋升为新的根节点 [p.46]

5.7 InnoDB 引擎中 B+ 树参数 n 的计算 [p.33–35]

5.7.1 参数 n 的影响因素 [p.33]

- 并非固定，由存储页大小和键值类型动态决定 [p.33]
- 页大小 (Page Size): InnoDB 默认 16KB (可通过 `innodb_page_size` 参数调整) [p.33]
- 键值类型: 主键 (INT/BIGINT) 和附加指针占用空间不同 [p.33]

5.7.2 单个键的存储开销 [p.34]

- 主键 (INT): 4 字节
- 子节点指针 (FIL_PAGE_OFFSET): 6 字节
- 其他元信息 (事务 ID 等): 约 10 字节
- 总计: 每个键 + 指针约占用 20 字节 [p.34]

5.7.3 单页键容量与树高 [p.34–35]

- 有效数据空间: 约 15KB (扣除页头、页尾元数据) [p.34]
- 最大键数 $n \approx 15\text{KB}/20\text{B} \approx 750$ [p.34]
- 实际实现中, InnoDB 的 B+ 树节点 (页) 通常可存储几百到上千个键 [p.34]
- 若主键为 INT (4 字节), 每页可存更多键 ($n \approx 1024$), 4 层可存约 $1024^3 \times 16 \approx 170$ 亿行 [p.34]
- 为什么需要这么大的 n ? [p.35]:
 - 减少树高: $n = 750$, 1 亿条记录时树高仅需 3 层 [p.35]
 - 降低磁盘 I/O: 树高越小, 磁盘读取次数越少 (通常 1–3 次 I/O 即可定位数据) [p.35]
- 优化建议 [p.35]:
 - 主键尽量短 (用 INT(4 字节) 而非 UUID(16 字节)), 增加 n , 降低树高

- 避免过长索引（如 VARCHAR(255)），减少单页键数量

6 MySQL 中的索引 (MySQL Index) [p.47–54]

6.1 索引的逻辑分类 [p.47]

以 InnoDB 引擎为例，索引分为聚集索引和非聚集索引。从逻辑上，索引可分为：

1. **普通索引 (Normal Index)** [p.47]：最基本的索引类型，无任何限制，唯一任务是加速数据访问。允许在定义索引的列中插入重复值和空值。
2. **唯一索引 (Unique Index)** [p.47]：与普通索引类似，但目的是避免数据重复。列值必须唯一，允许有空值。若是组合索引，列值组合必须唯一。使用 UNIQUE 关键字创建。
3. **主键索引 (Primary Key Index)** [p.47]：专门为主键字段创建的索引，是一种特殊的唯一索引，不允许值重复或为空。使用 PRIMARY KEY 关键字，不能使用 CREATE INDEX 语句创建。
4. **空间索引 (Spatial Index)** [p.47]：对空间数据类型字段建立的索引，主要用于地理空间数据类型，很少用到。
5. **全文索引 (Fulltext Index)** [p.47]：用于查找文本中的关键字，只能在 CHAR、VARCHAR 或 TEXT 类型的列上创建。MySQL 中只有 MyISAM 引擎支持全文索引（注：InnoDB 在 MySQL 5.6+ 也已支持）。

6.2 单列索引与复合索引 [p.48–49]

6.2.1 单列索引 (Single-Column Index) [p.48]

- 索引只包含原表的一个列，只根据该字段进行索引 [p.48]
- 例：在 student 表的 address 字段上建立名为 index_addr 的单列索引
- 可使用前缀索引：CREATE INDEX index_addr ON student(address(4)); 只索引 address 字段前 4 个字符 [p.48]

6.2.2 复合索引 / 多列索引 (Composite Index) [p.49]

- 将表的多个列共同组成一个索引 (Multiple-Key Index) [p.49]
- **最左前缀原则 (Leftmost Prefix)**：只有查询条件使用了索引中第一个字段时，索引才会被使用 [p.49]
- 例：CREATE INDEX index_na ON tb_student(name,address); 查询条件中必须有 name 字段才能使用索引 [p.49]
- **隐藏的多索引效果**：一个组合索引 (c1, c2, c3) 在查询中实际可提供三个加速索引 [p.49]：
 1. 单列索引 (c1)
 2. 双列索引 (c1, c2)
 3. 三列索引 (c1, c2, c3)

6.3 索引的查看与创建 [p.50–52]

6.3.1 查看索引 [p.50]

- `SHOW INDEX FROM < 表名 >`; [p.50]

6.3.2 创建索引 [p.51–52]

- **CREATE INDEX 直接创建** [p.51]:

```
CREATE [UNIQUE] INDEX < 索引名 > ON < 表名 > (< 列名 > [< 长度 >] [ASC|DESC])
```

- < 索引名 >: 一个表可创建多个索引, 每个索引名在表中唯一
- < 表名 >: 要创建索引的表
- < 列名 >: 通常在 JOIN 和 WHERE 子句中经常出现的列作为索引列
- < 长度 >: 可选项, 使用列的前 length 个字符创建索引, 节省空间。BLOB 或 TEXT 类型必须使用前缀索引。MyISAM/InnoDB 最大上限为 1000 字节 [p.51]
- ASC|DESC: 指定升序或降序排列, 默认 ASC

- **建表时创建** [p.52]:

```
CREATE TABLE student(id INT NOT NULL,  
    name CHAR(45) DEFAULT NULL,  
    INDEX(name));
```

- **ALTER TABLE** [p.52]:

```
ALTER TABLE student ADD INDEX index_name (name);
```

- 普通索引示例: `CREATE INDEX index_name ON student(name)`; [p.52]
- 唯一索引示例: `CREATE UNIQUE INDEX index_name ON student(name)`; [p.52]

6.4 索引失效的场景 [p.53]

- **违反最左前缀原则** [p.53]:

- 单字段有索引, WHERE 条件使用多字段 (含带索引字段和非索引字段): 不使用索引
- 多字段组合索引, WHERE 条件中不包含第一个字段: 索引不生效

- **操作符导致索引失效**: `<` (不等于)、NOT、IN、NOT EXISTS 等条件, 查询优化器可能倾向于全表扫描 [p.53]

- **OR 条件**: 条件中有 OR, 即使部分条件带索引也不会使用索引。解决方案: 将 OR 条件中的每个列都加上索引 [p.53]

- **后置通配符走索引, 前置通配符失效** [p.53]:

- 走索引: `SELECT * FROM student WHERE name LIKE ' 张%'`; (后置通配符)
- 全表扫描: `SELECT * FROM student WHERE name LIKE '% 东'`; (前置通配符)

6.5 索引的正确使用 [p.54]

6.5.1 索引的优点 [p.54]

- 通过创建唯一索引保证数据库表中每行数据的唯一性 [p.54]
- 大大加快数据的查询速度——使用索引最主要的原因 [p.54]
- 加速表与表之间的连接 (JOIN) [p.54]
- 显著减少分组 (GROUP BY) 和排序 (ORDER BY) 的时间 [p.54]

6.5.2 索引的缺点 [p.54]

- 创建和维护索引组需耗费时间，且随数据量增加而增加 [p.54]
- 索引需要占用磁盘空间——除数据表空间外，每个索引还需额外物理空间 [p.54]
- 当对表数据进行增删改时，索引也要动态维护，降低了数据的维护速度 [p.54]

7 散列索引 (Hash Index) [p.55–77]

7.1 散列基本概念 [p.55]

- 桶 (Bucket): 包含一个或多个记录的存储单元（通常是一个磁盘块） [p.55]
- 散列文件组织 (Hash File Organization): 使用散列函数直接从记录的搜索码值获得记录的桶地址 [p.55]
- 散列函数 h : 从所有搜索码值集合 K 到所有桶地址集合 B 的函数 [p.55]

$$h : K \rightarrow B$$

- 散列函数用于定位记录进行访问、插入和删除 [p.55]
- 不同搜索码值可能映射到相同桶，因此整个桶需要顺序搜索来定位特定记录 [p.55]

7.2 散列函数示例 [p.56–58]

7.2.1 示例：以 dept_name 为键的散列文件 [p.56–57]

- 有 10 个桶 [p.56]
- 第 i 个字符的二进制表示假定为整数 i
- 散列函数返回所有字符二进制表示之和模 10 [p.56]:

$$h(\text{dept_name}) = \left(\sum \text{char_binary_values} \right) \bmod 10$$

- 例: $h(\text{Music}) = 1$, $h(\text{History}) = 2$, $h(\text{Physics}) = 3$, $h(\text{Elec. Eng.}) = 3$ [p.56]

7.2.2 理想散列函数 [p.58]

- 最差的散列函数将所有搜索码值映射到同一个桶——访问时间与文件记录数成正比 [p.58]

- 理想散列函数是均匀的 (**Uniform**): 每个桶被分配相同数量的搜索码值 [p.58]
- 理想散列函数是随机的, 无论实际分布如何, 每个桶有相同数量的记录 [p.58]
- 典型散列函数对搜索码的内部二进制表示进行计算 [p.58]

7.3 桶溢出 (Bucket Overflow) [p.59–60]

7.3.1 溢出原因 [p.59]

- 桶数量不足 (Insufficient buckets) [p.59]
- 记录分布不均 (Skew in distribution of records) [p.59]:
 1. 多个记录具有相同搜索码值 (multiple records have same search-key value)
 2. 所选散列函数产生非均匀键值分布 (chosen hash function produces non-uniform distribution)
- 桶溢出概率可以降低但不能消除, 通过使用**溢出桶 (Overflow Buckets)** 处理 [p.59]

7.3.2 溢出处理方式 [p.60]

- **溢出链 (Overflow Chaining)**: 给定桶的溢出桶通过链表链接在一起 [p.60]
- 此方案称为**闭散列 (Closed Hashing)** [p.60]
- 另一种方式**开散列 (Open Hashing)** 不使用溢出桶, 但不适合数据库应用 [p.60]

7.4 散列索引 (Hash Index) [p.61–62]

- 散列不仅可用于文件组织, 也可用于创建索引结构 [p.61]
- **散列索引**将搜索键及其关联的记录指针组织成散列文件结构 [p.61]
- 严格来说, 散列索引始终是辅助索引 (Secondary Indices) [p.61]
 - 如果文件本身使用散列组织, 则不需要在其上使用相同搜索码的单独主散列索引
- 但术语“散列索引”通常指代辅助索引结构和散列组织文件两者 [p.61]

7.5 静态散列的问题 (Static Hashing Problems) [p.63]

- 在静态散列中, 函数 h 将搜索码值映射到固定的 B 个桶地址集合 [p.63]
- 数据库随时间增长或收缩:
 - 若初始桶数太小且文件增长, 性能因过多溢出而下降
 - 若为预期增长分配空间, 初始时将浪费大量空间 (桶欠满)
 - 若数据库收缩, 同样浪费空间
- 一种解决方案: 定期用新散列函数重新组织文件——代价高, 干扰正常操作 [p.63]
- 更好的方案: 允许桶数量动态修改 (Dynamic Hashing) [p.63]

7.6 可扩展散列 (Extendable Hashing) [p.64–77]

7.6.1 基本原理 [p.64]

- 适用于增长和收缩的数据库，允许散列函数动态修改 [p.64]
- 散列函数生成大范围的值——通常是 b 位整数， $b = 32$ [p.64]
- 任何时候只使用散列函数的前缀 (Prefix) 来索引到桶地址表 [p.64]
- 设前缀长度为 i 位， $0 \leq i \leq 32$ [p.64]
- 桶地址表大小 $= 2^i$ ，初始 $i = 0$ [p.64]
- 实际桶数 $< 2^i$ （因为桶地址表中多个条目可指向同一桶）[p.64]
- i 的值随数据库大小变化而增减；桶的数量也随合并和分裂动态变化 [p.64]

7.6.2 数据结构 [p.65–66]

- 每个桶 j 存有一个值 i_j [p.66]
- 指向同一桶的所有条目在前 i_j 位上具有相同值 [p.66]
- 查找桶（含搜索码 K_j ） [p.66]:
 1. 计算 $h(K_j) = X$
 2. 使用 X 的前 i 高位作为桶地址表中的偏移量，按指针找到对应桶
- 插入记录 [p.66]: 查找桶，若有空间则插入；否则必须分裂桶 [p.66]

7.6.3 桶分裂算法 [p.67]

当插入记录导致桶 j 满时：

1. 情况 1: $i > i_j$ （多个指针指向桶 j ）：
 - 分配新桶 z ，设 $i_z = (i_j + 1)$
 - 更新桶地址表中原本指向 j 的后半部分条目，改为指向 z
 - 移除桶 j 中每条记录并重新插入（到 j 或 z ）
 - 重新计算 K_j 的新桶并插入记录（如桶仍满需进一步分裂）
2. 情况 2: $i = i_j$ （只有一个指针指向桶 j ）：
 - 若 i 达到上限 b 或本次插入分裂次数过多，创建溢出桶
 - 否则：递增 i ，桶地址表翻倍；将表中每个条目替换为指向相同桶的两个条目
 - 重新计算 K_j 的桶地址表条目。此时 $i > i_j$ ，使用情况 1 处理

[p.67]

7.6.4 删除与合并 [p.68]

- 删除键值：在桶中定位并移除 [p.68]
- 若桶变空则删除桶本身（相应更新桶地址表）
- 合并 (Coalescing)：只能与“伙伴”桶合并（具有相同 i_j 值和相同 $i_j - 1$ 前缀）[p.68]

- 桶地址表大小也可以减小，但该操作代价高，仅在桶数远小于表大小时执行 [p.68]

7.6.5 可扩展散列示例 [p.70–76]

- 初始散列结构：桶大小 = 2 [p.70]
- 插入”Mozart”、”Srinivasan” 和”Wu” 后 [p.71]
- 插入 Einstein 后 [p.72]
- 插入 Gold 和 El Said 后 [p.73]
- 插入 Katz 后 [p.74]
- 插入 11 条记录后 [p.75]
- 再插入 Kim 记录后的散列结构 [p.76]

7.6.6 可扩展散列的优缺点 [p.77]

- 优点 [p.77]:
 - 散列性能不随文件增长而下降 (Hash performance does not degrade with growth of file)
 - 最小空间开销 (Minimal space overhead)
- 缺点 [p.77]:
 - 需要额外一层间接寻址来找到目标记录 (Extra level of indirection)
 - 桶地址表可能变得非常大（超过内存容量）(Bucket address table may become very big)
 - 无法在磁盘上分配非常大的连续区域
 - 改变桶地址表大小是昂贵操作
- 线性散列 (Linear Hashing) 是替代机制 [p.77]:
 - 允许目录（等价于桶地址表）的增量增长
 - 代价是更多桶溢出 (at the cost of more bucket overflows)

8 有序索引 vs 散列索引对比 [p.78]

- 需考虑的因素 [p.78]:
 - 定期重组的代价 (Cost of periodic re-organization)
 - 插入和删除的相对频率 (Relative frequency of insertions and deletions)
 - 是否值得以最坏情况访问时间为代价优化平均访问时间
 - 预期的查询类型 (Expected type of queries)
- 散列：通常在检索具有指定键值的记录时更好 [p.78]
- 有序索引：如果范围查询常见，有序索引更优 [p.78]
- 实践中的选择 [p.78]:

数据库系统	索引支持
PostgreSQL	支持散列索引，但因性能差而不推荐使用
Oracle	支持静态散列组织，但不支持散列索引
SQL Server	仅支持 B+ 树

9 位图索引 (Bitmap Index) [p.79–83]

9.1 基本概念 [p.79]

- 位图索引是一种特殊类型的索引，设计用于在多个键上高效查询 [p.79]
- 关系中的记录假定从 0 开始顺序编号 [p.79]
- 给定编号 n ，必须能轻松检索记录 n （定长记录时特别容易） [p.79]
- 适用范围：取值较少（基数低）的属性 [p.79]:
 - 例：性别 (gender)、国家 (country)、州 (state)
 - 例：收入水平 (income-level): 0–9999, 10000–19999, 20000–50000, 50000+
- 位图 (Bitmap) 就是一个位数组 (Array of bits) [p.79]

9.2 位图索引结构 [p.80]

- 简化形式：属性每个值对应一个位图 [p.80]
- 位图包含与记录数相等的位数 (Bitmap has as many bits as records) [p.80]
- 对于值 v 的位图，若某记录具有属性值 v 则对应的位为 1，否则为 0 [p.80]

9.3 位图操作与查询 [p.81]

- 位图索引对于多属性查询特别有用，对单属性查询帮助不明显 [p.81]
- 使用位图操作 (Bitmap Operations) 回答查询 [p.81]:
 - 交集 (Intersection, AND)
 - 并集 (Union, OR)
 - 补集 (Complementation, NOT)
- 示例 [p.81]:

$$100110 \text{ AND } 110011 = 100010$$

$$100110 \text{ OR } 110011 = 110111$$

$$\text{NOT } 100110 = 011001$$

- 查询：收入水平 L1 的男性 $\rightarrow 10010 \text{ AND } 10100 = 10000$ （然后检索匹配的元组） [p.81]
- 计数匹配元组更快——无需检索记录，直接统计位图中 1 的个数 [p.81]

9.4 位图索引的空间效率 [p.82]

- 位图索引通常比关系大小小得多 [p.82]

- 若记录为 100 字节，单个位图空间为关系的 $1/800$ [p.82]
- 若属性有 8 个不同值，位图仅占关系大小的 1% [p.82]
- 删除处理 [p.82]:
 - 需要存在位图 (**Existence Bitmap**) 标记某个记录位置是否有有效记录
 - 补集操作需要: $\text{NOT}(A=v) = (\text{NOT bitmap}-A-v) \text{ AND ExistenceBitmap}$ [p.82]
 - 应为所有值保留位图，包括 null 值，以正确处理 SQL null 语义 [p.82]

9.5 位图计算优化 [p.83]

- 位图被打包到字 (Word) 中；单条 CPU AND 指令可一次计算 32 或 64 位 [p.83]
- 例：100 万位位图的 AND 仅需 31,250 条指令 [p.83]
- 快速计数 1 的个数：使用每个字节索引到 256 元素的预计算数组，每个元素存储该字节中 1 的个数 [p.83]
- 可使用字节对进一步加速（更高内存代价） [p.83]
- 位图集成到 B+ 树：对于有大量匹配记录的值，可在 B+ 树叶节点层用位图代替元组 ID 列表 [p.83]
 - 当超过 $1/64$ 的记录具有该值时值得使用（假设元组 ID 为 64 位）
 - 此技术融合了位图和 B+ 树索引的优势 [p.83]

10 SQL 索引语法 (SQL Index Syntax) [p.84]

- 创建索引 [p.84]:

```
CREATE [UNIQUE] INDEX <index-name> ON <relation-name> (<attribute-list>);
```

- 例：CREATE INDEX b-index ON branch(branch_name); [p.84]
- 创建唯一索引：间接指定并强制搜索码是候选码的条件 [p.84]
- 删除索引 [p.84]:

```
DROP INDEX <index-name>;
```

- 大多数数据库系统允许指定索引类型和聚簇 [p.84]

11 章末总结：核心考点速查表

考点	要点	页码
索引定义	加速数据检索的数据结构，避免全表扫描	p.1
索引基本类型	Ordered Indices vs Hash Indices	p.4

考点	要点	页码
索引评估指标	访问类型、时间、插入/删除时间、空间开销	p.6
主索引 vs 辅助索引	主索引 = Clustered (文件物理顺序), 辅助索引 = Non-clustered	p.7
稠密 vs 稀疏索引	稠密 = 每个值都有条目; 稀疏 = 每块一个条目	p.8–12
多级索引	外层索引 (稀疏, 放内存) + 内层索引	p.14–15
B 树 vs B+ 树	B+ 树数据仅在叶节点, 叶节点有链表	p.20–24
B+ 树形式定义	所有路径等长; 非叶节点 $\lceil n/2 \rceil - n$ 个子节点	p.27
B+ 树树高	$\lceil \log_{\lceil n/2 \rceil}(K) \rceil$	p.32, p.37
B+ 树查询复杂度	100 万键值 $n = 100$ 时最多 4 次节点访问 vs 二叉树 20 次	p.37
B+ 树插入	叶子分裂 (向上传播); 非叶分裂 (中间键上升)	p.38–42
B+ 树删除	欠满处理: 借调 (Borrow) / 合并 (Merge)	p.43–46
InnoDB B+ 树参数	$n \approx 750$ (16KB 页, INT 主键), 3–4 层可存 170 亿行	p.33–35
MySQL 索引类型	普通、唯一、主键、空间、全文	p.47
复合索引	最左前缀原则; 一个 (c1,c2,c3) 提供三个索引	p.49
索引失效	违反最左前缀; $\langle \rangle$, OR, NOT, 前置通配符 LIKE	p.53
索引优缺点	加速查询、排序、JOIN vs 空间开销、维护开销	p.54
散列函数	$h : K \rightarrow B$; 理想散列函数均匀分布	p.55, p.58
桶溢出处理	溢出链 (Overflow Chaining) = 闭散列	p.59–60
静态散列问题	固定桶数不适应数据量变化; 需定期重组	p.63
可扩展散列	前缀索引 + 桶地址表 + 动态分裂/合并	p.64–68
可扩展散列优缺点	性能不降 vs 额外间接层、表可能过大	p.77

考点	要点	页码
线性散列	目录增量增长，代价 = 更多桶溢出	p.77
有序索引 vs 散列	等值查询选散列；范围查询选有序索引	p.78
位图索引	低基数属性；多属性 AND/OR/NOT 查询；空间 < 关系 1/800	p.79-82
位图计算优化	单指令 32-64 位并行；COUNT 用 256 预计算数组	p.83
SQL 创建索引	CREATE INDEX ... ON ... / DROP INDEX	p.84